09 / 07 — Modern React APIs: Actions, `use`, Optimistic UI, Refs, Metadata, and Resource Hints in Depth

1. Lesson metadata

Level	Advanced
Study time	5–6 hours (reading 2 h, experiments 2 h, exercises 1.5 h)
Prerequisites	Phase 3 / 22 — Modern hooks and APIs, Phase 3 / 21 — Forms and complex state, Phase 6 / 05 — Optimistic updates, 06 — Concurrent rendering in practice, 03 — Server Components in depth, 04 — Boundaries and serialization
Next	08 — React Compiler
Related	13 — Forms, actions and mutations, 11 — Progressive enhancement, 15 — React API changes across versions

> **Version notice (checked 2026-10-22 against React 19.3.1 and the react.dev 19.3 reference).** This lesson targets the React 19.x line. The React 19.0 APIs here (Actions, `useActionState`, `useOptimistic`, `use`, ref as prop, `<Context>` as provider, metadata and resource hints) are stable. `useEffectEvent`, `<Activity>`, `cacheSignal` and the Performance Tracks arrived stable in **19.2**; `<ViewTransition>`, `addTransitionType`, Fragment Refs and `browser()` became stable in **19.3** (they were experimental/Canary before). Your installed version still decides what exists: check `React.version`. `ReactDOM.render`, `forwardRef`, `defaultProps` on function components, `propTypes`, legacy context, and string refs are historical and are only mentioned for migration.

What this lesson is and is not. Lesson 22 in Phase 3 introduced these APIs one by one with the happy path. Here we study their **semantics, composition, and failure modes**: exactly what is guaranteed, what the type signatures imply, how Actions queue and reset, how optimistic state reverts, how `use` interacts with Suspense and error handling, what the ref and context changes mean for libraries, what the metadata/resource APIs do on server and client, and how all of this composes into a coherent mutation architecture. We deliberately do not re-explain basic syntax.

Learning objectives. After this lesson you can:

1. State precisely what an *Action* is, what React tracks for it (pending, error, form reset), and how it differs from an event handler.
2. Use `useActionState`, `useFormStatus`, and `useOptimistic` together in one form and predict the render sequence, including failure.
3. Use `use` correctly with promises and context, and explain its restrictions (stable promises, no try/catch, conditional calls).
4. Migrate components and TypeScript types for ref-as-prop and ref callback cleanup, and context-as-provider.
5. Use document metadata and resource-loading APIs (`<title>`, `<link rel="stylesheet" precedence>`, `preload`, `preinit`, `preconnect`) and predict their placement and deduplication.
6. Apply `useEffectEvent` and `<Activity>` (stable since 19.2) appropriately and know their constraints.
7. Choose between Actions + Server Functions, client-side mutation libraries, and imperative handlers.

8. Test and debug Action-based UIs.

Concepts covered: Action (async transition), `<form action>`, `formAction`, `FormData`, automatic form reset, `requestFormReset`, `useActionState` signature and queueing, `permalink`, `useFormStatus` scoping rules, `useOptimistic` lifecycle and reducer form, `use(promise)` semantics, `use(context)`, ref as prop, ref cleanup, TS implications, `<Context value>`, hoisted metadata, stylesheet precedence, resource hints, `useDeferredValue` `initialValue`, `useEffectEvent`, `<Activity>`, error reporting root options.

---

2. Deep conceptual explanation

2.1 Why these APIs exist: mutation as a first-class concept

Before React 19, mutation flows were assembled by hand in every app: an event handler calls `fetch`, flips `isLoading`, catches errors into state, updates a cache, and resets the form. Each team built a slightly different state machine, usually with bugs (double submit, stuck spinners, lost errors, no cancellation of stale results). React 19 elevates the pattern into the framework:

> An **Action** is *a function (sync or async) that you pass to React to run inside a transition*, so that React can track **pending**, handle **errors** via boundaries, **batch** the resulting updates, and coordinate **optimistic** state and **form reset**.

The conceptual payoffs:

Concern	Hand-rolled	Action-based
Pending state	`useState(false)` + try/finally	`isPending` from `useTransition` / `useActionState` / `useFormStatus`
Error handling	`catch` → state	Error boundary (thrown) or returned state (`useActionState`)
Optimistic UI	Temporary state + manual rollback	`useOptimistic` auto-reverts when the Action settles
Form reset	Manual `reset()`	Automatic after a successful form Action (uncontrolled fields)
Double submit	Disabled flag	Sequencing/queueing by React
Works before JS loads	No	Yes with Server Functions + framework (progressive enhancement)

2.2 Anatomy: where Actions plug in

An Action is invoked through one of three doors:

1. `startTransition(async () => { … })` / the function from `useTransition()`: the generic door for event handlers.
2. `<form action={fn}>` (and `<button formAction={fn}>`): React passes `FormData` to `fn`, runs it in a transition, then resets the form on success.
3. `useActionState(fn, initial)`: wraps `fn` so its *return value becomes state*, and gives you `isPending`; the returned `formAction` is used as a form `action`.

All three share the runtime: *the function runs inside a transition scope;* `isPending` *is true until it settles and resulting updates commit*. That is why Actions inherit the transition behavior from 06 §3.5, including the caveat that updates after `await` need re-wrapping.

2.3 Optimistic state as a *view*, not as truth

`useOptimistic(value)` returns a derived value that *temporarily overlays* a pending change on top of the real state. The overlay exists only while an Action is in progress; when the Action settles (success or failure), React discards the overlay and renders the **real** state, which on success should already contain the confirmed change (because your Action updated it) and on failure simply doesn't. This inverts the usual approach (apply change, remember to undo): the default outcome is the *truth*, the optimistic view is the exception. Your obligation is to make the Action update the real state (or revalidate server data) before it settles.

2.4 `use` as a "read" primitive

`use(resource)` reads a value from a resource: a **promise** (suspends until resolved; throws on rejection to an error boundary) or a **context** (like `useContext`, but callable conditionally and after early returns). It is called "an API, not a Hook" because it can be used in conditionals and loops, but it still must be called **during render** of a component or a Hook.

Resource	Behavior	Restriction
Promise	Suspends until fulfilled; returns value; on rejection, throws to nearest error boundary	Must have **stable identity** across renders; a promise created in a Client Component's render is not supported (new each render): React keeps showing the fallback on every re-render (react.dev)
Context	Returns the nearest provider value; can be conditional	Same propagation semantics as `useContext`

Crucially, `use` **must not be wrapped in** `try/catch`, because suspension is implemented by throwing an internal exception: react.dev says it "cannot be wrapped in try-catch", and React 19.3.1 logs in development "`use` was called from inside a try/catch block. This is not allowed and can lead to unexpected behavior" (executed; in that run the `catch` did intercept the rejection and rendered its fallback, which is precisely the unsupported behavior the warning refers to). Use error boundaries, or attach `.catch` to the promise to produce a fallback value *before* passing it to `use`.

2.5 Misconceptions

Misconception	Reality
"Actions are for forms only."	Any async mutation can run in a transition; forms are the most ergonomic entry.
"`useOptimistic` stores state."	It's a *derived overlay*; real state lives elsewhere.
"`useFormStatus` works in the component that renders the `<form>`."	It reads the status of a **parent** `<form>`; call it in a child component rendered inside the form.
"`useActionState` and `useState` + handler are interchangeable."	`useActionState` adds queueing, pending tracking, progressive-enhancement hooks (`permalink`), and integration with form reset.
"Ref callbacks can return anything."	In React 19, a returned function is treated as **cleanup**; implicit returns from arrow functions (e.g., `ref={el => (inst = el)}`) are an error in TS typings (verified with `tsc` against `@types/react` 19) and a behavior change.

Misconception	Reality
"`<Context.Provider>` is removed."	`<Context value>` is the new form; `.Provider` still works; react.dev's `createContext` page calls it "a legacy way to provide the context value before React 19" and the React 19 post says it will be deprecated in a future version (no removal date announced).
"`forwardRef` is required for refs."	Function components receive `ref` as a prop in React 19; `forwardRef` remains for compatibility; react.dev says it "will be deprecated in a future release".
"`useEffectEvent` removes the need for dependency arrays."	It lets effects read latest values without re-running; it doesn't change what belongs in dependencies for other values; react.dev warns not to use it "to avoid specifying dependencies".

3. Internal mechanics

> **Labels. Public guarantee** documented on react.dev; **Teaching model**; **Implementation detail** may change.

3.1 Public guarantees

1. `<form action={fn}>`: when submitted, React calls `fn(formData)` inside a transition (the form is *not* submitted via the browser's default navigation when JS is active; with Server Functions and a framework, a no-JS submission falls back to a regular form POST).
2. After a form Action's function settles successfully, React resets **uncontrolled** form fields automatically (react.dev: "After the action function succeeds, all uncontrolled field elements in the form are reset"). `react-dom` 19.3.1 also exports `requestFormReset` and ships its TypeScript types, but there is no react.dev reference page for it (the URL 404s on 2026-10-22), so treat it as lightly documented; for manual control, react.dev's `<form>` page uses `formRef.current.reset()` in the optimistic example.
3. `useActionState(action, initialState, permalink?)` returns `[state, formAction, isPending]`. The action is called as `action(previousState, payload)` where `payload` is the `FormData` when used as a form action. Its return value (awaited) becomes the new `state`. Calls are queued and executed one at a time. `dispatchAction` must be called from an Action (a form action prop or inside `startTransition`), otherwise React logs an error in development; if the action *throws*, React cancels all queued actions and shows the nearest Error Boundary (react.dev). (`react-dom` still exports the old name `useFormState` as an alias; use `useActionState` from `react`.)
4. `useFormStatus()` (from `react-dom`) returns `{ pending, data, method, action }` for the *nearest parent* `<form>` and does not report forms rendered by the same component.
5. `useOptimistic(state, updateFn?)` returns `[optimisticState, addOptimistic]`; `addOptimistic` must be called **inside an Action/transition** (otherwise React 19.3.1 logs "An optimistic state update occurred outside a transition or action. To fix, move the update to an action, or wrap with startTransition."); the optimistic state reverts to `state` when the Action ends.
6. `use(promise)` suspends; `use(context)` reads context; both callable conditionally; neither inside try/catch. React 19.3 adds a DEV-only warning when a component appears to have been unblocked by a conditional `use()` (19.3 changelog; I could not trigger it in a simple test, so the exact trigger is unverified).
7. In React 19 `ref` is a regular prop for function components; ref callbacks may return a cleanup function; `Context` objects are renderable as providers.
8. `<title>`, `<meta>`, `<link>` rendered anywhere are hoisted into `<head>`; `<link rel="stylesheet" href precedence>` participates in stylesheet ordering and loading integration (react.dev `<Suspense>`: React

blocks the boundary until the stylesheet loads, up to a timeout); `<script async src>` is deduplicated.
9. `react-dom` exports `preload`, `preinit`, `preconnect`, `prefetchDNS`, `preloadModule`, `preinitModule` for resource hints that React deduplicates and emits during SSR early.
10. React 19 root options `onCaughtError`, `onUncaughtError`, `onRecoverableError` centralize error reporting.

3.2 Teaching model: the Action queue

```
User submits → useActionState dispatches payload P1
   queue: [P1]       state S0
   run action(S0, P1) …(async)… isPending = true
User submits again (P2) before P1 resolved:
   queue: [P1(running), P2]
   P1 resolves → state S1 committed (or batched) → run action(S1, P2)   // P2 sees S1 as prevState
   P2 resolves → state S2; isPending = false
```

Implications:

- **Sequential semantics:** each Action sees the state produced by the previous one: you can build correct counters/reducers without races among *your own submissions*.
- **Latency stacks:** queued actions wait; a slow first action delays the second. If submissions should be independent (e.g., many parallel uploads), model them as separate state instances or use a different mechanism.
- `isPending` **is true while any queued action is outstanding.**
- Batching is documented (react.dev, `useActionState` / `useTransition` caveats: "If there are multiple ongoing Actions, React batches them together. This is a limitation that may be removed in a future release"). Executed on 19.3.1 with three queued actions on one `useActionState`: renders logged `count=0 pending=true` three times and then a single `count=3 pending=false`; the intermediate counts 1 and 2 were never rendered. Do not assume per-action commits.

3.3 Teaching model: the render sequence of an optimistic form

Form with `useActionState` + `useOptimistic` adding a todo:

Step	Event	Real state `todos`	Optimistic view	`isPending`	DOM shows
0	idle	[A]	[A]	false	A
1	submit "B": the Action begins; `addOptimistic("B")` called synchronously inside the Action	[A]	[A, B*]	true	A, B (pending style)
2	`await server.add("B")` in flight	[A]	[A, B*]	true	A, B*
3 success: Action calls `setTodos` /revalidate to include B before returning	[A, B]	overlay discarded when Action settles: view = real	→ [A, B]	false (after commit)	A, B (confirmed)

Step	Event	Real state `todos`	Optimistic view	`isPending`	DOM shows
3′ failure: Action throws or returns error state	[A]	overlay discarded	[A]	false	A (B vanishes; show error from returned state)

Predict: if step 3 forgets to update `todos`, what does the user see after success? *Answer:* B disappears (overlay ends, real state lacks B), then maybe reappears after a refetch: a flicker. The optimistic overlay must be backed by a real-state update (state setter, query cache update, or server revalidation that commits before the Action ends).

3.4 Teaching model: `use(promise)` loop hazard

```
function Bad({ id }: { id: string }) {
  const data = use(fetch(`/api/${id}`).then(r => r.json()));   // new promise every render
  return <p>{data.name}</p>;
}
```

Render 1 creates promise P1 → suspends. P1 resolves → React retries render → creates P2 (a different pending promise) → suspends again → … never settles (executed on 19.3.1: ~47 renders in 500 ms with a 10 ms promise, fallback never replaced). react.dev: "If a new Promise is created directly in render, React will display the Suspense fallback on every re-render." React has a dev warning ("A component was suspended by an uncached promise…") but it only fires in some replay situations and did not fire in my loop test, so don't depend on it. In Server Components `use` / `await` of a freshly created promise is fine because the server render is **not re-executed** for each retry the same way (the component function continues from where the async function awaited); but "create promise → `use`" is still preferred only for passing down; `await` is more natural in async server components (03 §3.8).

3.5 Teaching model: refs as props and cleanup

React 18 (legacy for this): function components couldn't receive `ref`; `forwardRef` wrapped them. React 19: `ref` appears in props.

```
function Input({ ref, ...props }: React.ComponentProps<"input">) {   // ref: Ref<HTMLInputElement> | undefined
  return <input ref={ref} {...props} />;
}
```

Ref callback lifecycle in React 19:

Event	Callback behavior
Mount / ref attached	`callback(node)` called; if it returns a function, that is stored as cleanup
Unmount / ref detached	If cleanup was returned, call **cleanup** (and do not call `callback(null)`); otherwise `callback(null)` as in React 18
Callback identity changes between renders	Old cleanup (or `callback(null)`) then new `callback(node)`

TS note: with `@types/react` for React 19, ref callbacks returning anything other than a cleanup function or `void` are type errors (verified with `tsc --noEmit`), flagging the classic bug `ref={(el) => (nodes[i] = el)}` whose assignment expression value (the element) would now be misinterpreted as a cleanup. Use braces: `ref={(el) => { nodes[i] = el; }}`. The React 19 upgrade guide documents a codemod for this, `no-implicit-ref-callback-return`, part of the `react/19/migration-recipe`. Executed lifecycle: mount → `ref(el)`; identity change → cleanup of the old callback, then `ref(el)` of the new one; unmount → cleanup only, no `ref(null)` call; a callback that returns nothing still gets `ref(null)` on unmount.

3.6 Teaching model: metadata and resources on the server vs client

API	Server (streaming SSR)	Client
`<title>`, `<meta>`, `<link>` in a component	Hoisted into `<head>` of the response; elements discovered in suspended boundaries after the head has flushed are emitted with that content; the exact mechanism is internal	Hoisted into `document.head` on commit
`<link rel="stylesheet" precedence="x">`	Placed in head by precedence; deduped by `href`; React can delay revealing a boundary until its stylesheet is available to avoid FOUC (react.dev `<Suspense>`: "React blocks the boundary until the stylesheet loads, up to a timeout")	Inserts the link; the boundary waits for the load (same timeout rule)
`preload(href, {as})`	Emitted as `<link rel="preload">` early in the shell	Inserted into head
`preinit(href, {as})`	Emits the *resource itself* (script/style) eagerly	Loads/executes now
`preconnect(origin)` / `prefetchDNS`	`<link rel="preconnect">` / `dns-prefetch`	Same

Because React deduplicates by URL, calling `preload` from many components is safe. Since this is **render-phase** side-effect-like but idempotent, react.dev documents all of `preload`, `preinit`, `preconnect` and `prefetchDNS` as callable while rendering (and, in the browser, also from effects and event handlers); during server rendering they only take effect when called while rendering a component or in an async context originating from one.

3.7 `useEffectEvent`: what it formalizes

An Effect Event is a function that always sees the **latest props/state** when called but is *not reactive*: it is excluded from the effect's dependency list and must be called **only inside effects** (`useEffect`, `useLayoutEffect`, `useInsertionEffect`) and other effect events of the same component, not during render (React throws: "A function wrapped in useEffectEvent can't be called during rendering"; executed), not passed to other components or hooks, not stored in dependency arrays. Executed on 19.3.1: changing only `theme` re-rendered without reconnecting, and the effect event read the latest `theme`; changing `roomId` ran cleanup then setup, and the event saw the new `roomId`.

```
function Chat({ roomId, theme }: { roomId: string; theme: "light" | "dark" }) {
  const onConnected = useEffectEvent(() => { showToast(`Connected to ${roomId}`, theme); });
  useEffect(() => {
    const conn = createConnection(roomId);
    conn.on("connected", () => onConnected());
    conn.connect();
    return () => conn.disconnect();
  }, [roomId]);                    // theme changes don't reconnect; onConnected reads latest
theme
}
```

`useEffectEvent` shipped as stable in React 19.2 after a long experimental period; use `eslint-plugin-react-hooks` v6 or later, whose rules enforce the call-site restrictions (it also errors if you put the event in a dependency array). It's the sanctioned replacement for the "ref holding the latest callback" pattern (Phase 3 / 05).

3.8 `<Activity>`: hide with state preserved

`<Activity mode="visible" | "hidden">` (default `"visible"`; React 19.2 supports exactly these two modes, more are planned) lets part of the tree be hidden **without unmounting**: state is preserved; the DOM gets `display: none`; Effects are destroyed while hidden (subscriptions don't run for hidden UI) and re-created on show; hidden children still re-render for new props, at lower priority. A hidden Activity whose children render only text (no DOM element) renders nothing. Executed on 19.3.1: on mount a hidden pane rendered (`S render n=0`) but none of its effects ran; hiding the visible pane logged cleanup for all its effects and kept its state (`F1` after a click); showing it again re-ran its effects with state intact. Use cases: preserving tab/route state, pre-rendering likely-next screens at low priority. Implication for analytics: effects may run again when revealed; make them idempotent.

---

4. Step-by-step execution walkthrough

4.1 Comment form with all three hooks, success path

```
// Server Function (or client function) returns the new state of the thread
async function postComment(prev: ThreadState, formData: FormData): Promise<ThreadState> { /* … */ }
```

1. **Mount:** `useActionState(postComment, initial)` → `[thread, formAction, isPending=false]`. `useOptimistic(thread.comments, (cur, text: string) => [...cur, { id: "tmp", text, pending: true }])` → `[optimisticComments, addOptimistic]`.
2. **Submit:** the browser `submit` event; React intercepts (JS active), builds `FormData`, calls the wrapped action in a transition.
3. **Inside the wrapper** *you* (in a client-side `action` prop that calls both) do:

```
async function submit(formData: FormData) {
  const text = String(formData.get("text") ?? "");
  addOptimistic(text);                  // sync: part of the Action → optimistic overlay
renders now
  await formAction(formData);           // dispatch to useActionState queue
}
<form action={submit}>…</form>
```

Executed on 19.3.1 (`addOptimistic` then `await formAction(formData)` inside a wrapper form action): during the Action the list showed `hi*` with `isPending=true`; after resolve one render committed `state=hi`, `opt=hi`, `isPending=false`. react.dev's `useActionState` page has a "Using with useOptimistic" section for the same arrangement.

4. **Render (urgent for overlay):** `optimisticComments` includes the temp comment; `isPending` true; the child `SubmitButton` calling `useFormStatus()` sees `pending=true` and disables itself.
5. **Server returns** new `ThreadState` including the real comment id.
6. **Commit:** state updates; overlay discarded; `isPending=false`; uncontrolled `text` input auto-reset.

4.2 Failure path

At step 5 the function returns `{ ...prev, error: "Rate limited" }` (as a **value**). State commits with `error`; overlay is discarded (the temp comment disappears); the form fields **are still reset** after Action completion (documented default for form Actions); you may want to preserve user input by returning it in state and using it as `defaultValue` (§5.3).

If the function **throws** instead, the error propagates to the nearest error boundary during the transition's commit; the overlay is discarded.

---

5. Code examples

5.1 Minimal: `useActionState` with a plain async function

```tsx
"use client";
import { useActionState } from "react";

type State = { count: number; error?: string };

async function increment(prev: State, formData: FormData): Promise<State> {
  const by = Number(formData.get("by") ?? 1);
  if (!Number.isFinite(by)) return { ...prev, error: "Not a number" };
  await new Promise((r) => setTimeout(r, 300));          // simulate latency
  return { count: prev.count + by };
}

export function Counter() {
  const [state, formAction, isPending] = useActionState(increment, { count: 0 });
  return (
    <form action={formAction}>
      <input name="by" defaultValue="1" inputMode="numeric" />
      <button disabled={isPending}>Add</button>
      <p>{state.count}{isPending ? " …" : ""}</p>
      {state.error && <p role="alert">{state.error}</p>}
    </form>
  );
}
```

Predict: click "Add" twice quickly. *Answer:* with `disabled={isPending}` as written, the first click's re-render disables the button, so the second click is ignored and one Action runs (count 1 after ~300 ms). If you remove `disabled`, two Actions are queued; the second sees the state produced by the first (`prev.count` includes the first increment); with 300 ms each, the final count is 2 after ~600 ms, and (executed on 19.3.1) React commits the state once, after the last queued Action, not once per Action. Note the **input resets** because it's uncontrolled and the form Action completed; with queued Actions the reset also happened once, after the last one (the input still held the typed value after the first Action resolved).

5.2 Practical: optimistic list with `useOptimistic` and correct revert

```tsx
"use client";
import { useOptimistic, useRef, useActionState, startTransition } from "react";

type Todo = { id: string; title: string; pending?: boolean };
type ListState = { todos: Todo[]; error?: string };

export function TodoList({ initial, addTodo }: {
  initial: Todo[];
  addTodo: (title: string) => Promise<Todo>;   // may be a Server Function; returns the saved todo
}) {
  const [state, dispatch, isPending] = useActionState(
    async (prev: ListState, title: string): Promise<ListState> => {
      try {
        const saved = await addTodo(title);
        return { todos: [...prev.todos, saved] };
      } catch {
        return { ...prev, error: "Could not add todo" };
      }
    },
    { todos: initial } as ListState,
  );

  const [optimisticTodos, addOptimistic] = useOptimistic(
    state.todos,
    (current: Todo[], title: string) => [...current, { id: `tmp-${crypto.randomUUID()}`, title, pending: true }],
  );
  const formRef = useRef<HTMLFormElement>(null);

  function submit(formData: FormData) {
    const title = String(formData.get("title") ?? "").trim();
    if (!title) return;
    addOptimistic(title);          // inside the form Action → allowed; overlay until the Action settles
    startTransition(() => dispatch(title));   // dispatch to the queue as part of this Action
  }

  return (
    <form action={submit} ref={formRef}>
      <ul>{optimisticTodos.map((t) => <li key={t.id} style={{ opacity: t.pending ? 0.5 : 1 }}>{t.title}</li>)}</ul>
      <input name="title" />
      <button disabled={isPending}>Add</button>
      {state.error && <p role="alert">{state.error}</p>}
    </form>
  );
}
```

Line notes:

- The reducer form of `useOptimistic` builds the overlay from the *current real list* and the action argument; the temp id is only for keys.
- Failure path: the reducer in `useActionState` returns the old todos plus `error`; the overlay vanishes: the item disappears and the error shows.
- Executed on 19.3.1 (success and failure): success rendered `real=A opt=A,B* pending=true` then one commit `real=A,B opt=A,B pending=false`; failure ended with `real=A,B opt=A,B` plus the error message. Several equivalent arrangements of `useOptimistic` with `useActionState` appear on react.dev; this one works. The invariant that matters: `addOptimistic` must be called **inside** an Action/transition, and the real state must update before the Action ends.

5.3 Practical: keep user input on validation failure

```
type FormState = { ok: boolean; values: { email: string }; errors: { email?: string } };

async function signup(prev: FormState, formData: FormData): Promise<FormState> {
  const email = String(formData.get("email") ?? "");
  if (!/^\S+@\S+\.\S+$/.test(email)) return { ok: false, values: { email }, errors: { email:
"Invalid email" } };
  await api.signup(email);
  return { ok: true, values: { email: "" }, errors: {} };
}

export function Signup() {
  const [state, action] = useActionState(signup, { ok: false, values: { email: "" }, errors: {} });
  return (
    <form action={action}>
      <input name="email" defaultValue={state.values.email} aria-invalid={!!state.errors.email} />
      {state.errors.email && <p role="alert">{state.errors.email}</p>}
      <SubmitButton />
    </form>
  );
}
```

Why `defaultValue={state.values.email}`: React resets the form after the Action; the reset restores `defaultValue`, which now holds the user's last input, so they don't lose their typing. Executed on 19.3.1: after an invalid submit typed `abc` came back as `ABC` when the action echoed an uppercased value into `defaultValue` (proving the reset re-applies the new `defaultValue`), and came back empty when the returned `defaultValue` was empty. `defaultValue` changes after mount only take effect on reset, which is exactly the mechanism used here.

5.4 Practical: `useFormStatus` in a design-system submit button

```tsx
"use client";
import { useFormStatus } from "react-dom";

export function SubmitButton({ children, pendingLabel = "Saving…" }: { children: React.ReactNode;
pendingLabel?: string }) {
  const { pending } = useFormStatus();         // status of the nearest PARENT <form>
  return (
    <button type="submit" disabled={pending} aria-disabled={pending}>
      {pending ? pendingLabel : children}
    </button>
  );
}
```

Design benefit: any form gets consistent pending UI without prop drilling `isPending`. Constraint: must render *inside* the form; calling `useFormStatus` in the component that renders `<form>` returns `pending: false` always.

5.5 Practical: `use` with context and conditional reads

```tsx
function Price({ amountCents, compact }: { amountCents: number; compact?: boolean }) {
  if (compact) return <span>{(amountCents / 100).toFixed(0)}</span>;     // early return BEFORE use
is OK for `use`
  const { locale, currency } = use(LocaleContext);                       // conditional context read
(impossible with useContext)
  return <span>{new Intl.NumberFormat(locale, { style: "currency", currency }).format(amountCents /
100)}</span>;
}
```

Benefit: the compact variant doesn't subscribe to the context at all, so it doesn't re-render on locale changes. This is a small runtime nicety; keep component logic readable.

5.6 Practical: promise props with `use` and error handling

```tsx
// Server component: starts the work; does not await it
const reviewsPromise = getReviews(productId).catch(() => [] as Review[]);   // convert failure to a
fallback value on the server...
return <Suspense fallback={<ReviewsSkeleton />}><Reviews reviewsPromise={reviewsPromise} />
</Suspense>;

// Client
"use client";
export function Reviews({ reviewsPromise }: { reviewsPromise: Promise<Review[]> }) {
  const reviews = use(reviewsPromise);            // stable promise from the server
  return reviews.length ? <ul>{reviews.map((r) => <li key={r.id}>{r.text}</li>)}</ul> : <p>No
reviews yet.</p>;
}
```

Design choice: converting rejection to `[]` hides failures. Alternatively let it reject to an error boundary with retry; choose based on whether partial UI is acceptable (05 §3.5).

5.7 Practical: ref as prop with cleanup, and a TS-correct ref callback

```
function VideoPlayer({ ref, src }: { ref?: React.Ref<HTMLVideoElement>; src: string }) {
  return <video ref={ref} src={src} />;
}

function Gallery({ items }: { items: string[] }) {
  const nodes = useRef(new Map<string, HTMLElement>());
  return (
    <ul>
      {items.map((id) => (
        <li
          key={id}
          ref={(el) => {                          // block body: returns void, not the element
            if (el) nodes.current.set(id, el);
            return () => { nodes.current.delete(id); };   // cleanup form (React 19): no `ref(null)`
call
          }}
        >
          {id}
        </li>
      ))}
    </ul>
  );
}
```

Notes: a fresh inline callback each render means React runs cleanup + setup on every render (cheap here). For heavy setup (observers), stabilize with `useCallback`. Cleanup-returning ref callbacks are React 19 only; libraries supporting older React must handle `null` calls.

5.8 Practical: `<Context value>` and metadata/resource APIs

```tsx
const ThemeContext = createContext<"light" | "dark">("light");

export function App() {
  return (
    <ThemeContext value="dark">          {/* React 19: no .Provider needed */}
      <ProductPage />
    </ThemeContext>
  );
}

import { preload, preconnect } from "react-dom";
function ProductPage({ product }: { product: Product }) {
  preconnect("https://images.example-cdn.com");                // deduped, idempotent hints in
render
  preload(product.heroUrl, { as: "image", fetchPriority: "high" });
  return (
    <article>
      <title>{`${product.name} | Shop`}</title>
      <meta name="description" content={product.summary} />
      <link rel="canonical" href={`https://shop.example/p/${product.slug}`} />
      <link rel="stylesheet" href="/css/product.css" precedence="default" />
      <h1>{product.name}</h1>
      <img src={product.heroUrl} alt={product.heroAlt} fetchPriority="high" />
    </article>
  );
}
```

Notes: the `<title>` is hoisted to `<head>` even though it is rendered deep in the tree; render only one `<title>` at a time: react.dev says that with more than one, React places all of them in `<head>` and "the behavior of browsers and search engines is undefined" (framework routers normally manage this for you). `precedence` groups stylesheets so ordering is deterministic and React can wait for load before revealing content depending on it. The `preload` call with `as: "image"` helps LCP when the image would otherwise be discovered late (e.g., behind Suspense).

5.9 Incorrect: `useFormStatus` in the form's own component

```tsx
// ❌
function Form() {
  const { pending } = useFormStatus();    // always false: this component renders the <form>; it is
not INSIDE one
  return <form action={save}><button disabled={pending}>Save</button></form>;
}
```

Corrected: move the button to a child component that calls `useFormStatus` (§5.4), or use `isPending` from `useActionState`.

5.10 Incorrect: optimistic update outside an Action; rollback by hand

```
// ❌
function Like({ liked }: { liked: boolean }) {
  const [optLiked, setOpt] = useOptimistic(liked);
  return <button onClick={async () => { setOpt(!liked); await api.like(); }}>…</button>;   // called
outside a transition
}
```

Problems: React warns ("An optimistic state update occurred outside a transition or action…"); the update is not tied to an Action, so it does not behave as a pending overlay (executed: the value did not show the optimistic state). **Corrected:**

```
<button onClick={() => startTransition(async () => { setOpt(!liked); await api.like(); /* then
update real state */ })}>
```

And make sure the real `liked` prop/state updates (via mutation + revalidation) before the Action completes; otherwise the UI flips back.

5.11 Incorrect: `use` in `try/catch` and in a handler

```
// ❌
function Profile({ p }: { p: Promise<User> }) {
  try { const u = use(p); return <h1>{u.name}</h1>; }
  catch { return <p>Failed</p>; }        // unsupported: React warns in DEV; catching interferes
with suspension signaling
}
function Button({ p }: { p: Promise<User> }) {
  return <button onClick={() => { const u = use(p); }} />;   // ❌ use only during render
}
```

Corrected: wrap with an error boundary; or `p.catch(() => fallback)` upstream; read in render and use the value in the handler.

5.12 Advanced: `useEffectEvent` for non-reactive logic inside an effect

```
function useDocumentTitleLogger(title: string, userId: string) {
  const log = useEffectEvent((t: string) => analytics.track("title_set", { title: t, userId }));
  useEffect(() => {
    document.title = title;
    log(title);                    // reads the latest userId without making the effect depend on it
  }, [title]);                     // effect re-runs only when `title` changes
}
```

Constraint reminders: only call `log` from inside effects/other effect events; don't pass it down; the lint plugin enforces. Misuse—using it to dodge dependencies that *should* re-trigger the effect—reintroduces stale-closure bugs; ask: "should this value cause the effect to re-run?" If yes, it's reactive.

5.13 Advanced: `<Activity>` preserving a tab's state

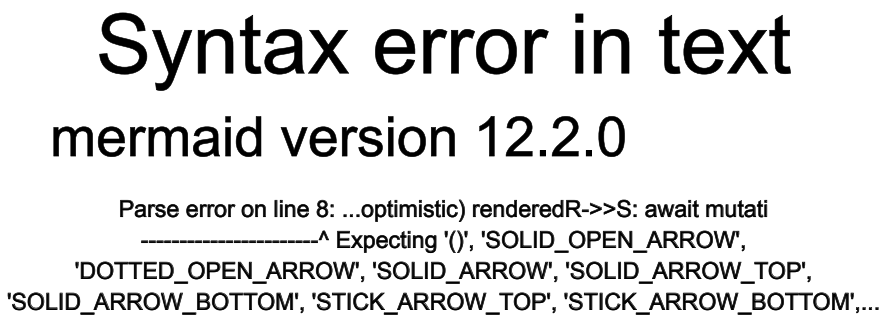
```tsx
function Tabs({ active }: { active: "feed" | "search" }) {
  return (
    <>
      <Activity mode={active === "feed" ? "visible" : "hidden"}><Feed /></Activity>
      <Activity mode={active === "search" ? "visible" : "hidden"}><Search /></Activity>
    </>
  );
}
```

Verified behavior (executed, see §3.8): hidden panes keep state (and the DOM node, so input/scroll state in it); effects in hidden panes are cleaned up and re-run on show; hidden renders are low priority. Cost: hidden subtrees stay in memory; avoid for many heavy panes.

---

6. Visual explanations

6.1 Action lifecycle



6.2 Which hook, which job

Need	Use
Pending/error/result state for a form mutation	`useActionState`
Pending UI in a child of a form	`useFormStatus`
Instant visual feedback that auto-reverts	`useOptimistic`
Pending for non-form async handler	`useTransition`
Read a promise/context in render	`use`
Latest values in effects without re-subscribing	`useEffectEvent`
Keep hidden UI's state	`<Activity>`

6.3 Optimistic overlay model

```
real state:      [A]  ──────────────────────────►  [A,B] (after Action commit)
overlay (B*):          ┌──────── Action in flight ────────┐
displayed:       [A]   [A, B*]                      [A, B]   (success)
                                                    [A]      (failure: overlay gone)
```

---

7. Alternative approaches and trade-offs

Criterion	Actions + `useActionState`	Event handler + `useState`	TanStack Query mutations	Redux Toolkit thunks	Server Function via `<form>`
Pending/error tracking	Built-in	Manual	Built-in (`isPending`, `error`)	Built-in (`createAsyncThunk` status)	Built-in (via hooks)
Optimistic UI	`useOptimistic`	Manual	`onMutate` rollback	Manual	`useOptimistic`
Cache invalidation	You/ framework	Manual	Built-in (`invalidateQueries`)	Manual/RTK Query	Framework revalidation
Works without JS	Yes with Server Functions + framework	No	No	No	Yes
Retry/backoff	Manual	Manual	Built-in	Manual	Manual
Offline/persisted mutations	No	No	Possible (persist plugins)	Custom	No
Concurrency control	Sequential queue	Manual	Configurable	Manual	Sequential
Learning curve	Medium	Low	Medium	Medium–High	Medium
Best for	Form-centric flows; RSC apps	Simple, local	Rich client apps with server state	Large global state apps	Progressive enhancement

Scenario where Actions are *not* best. A client-only app with complex server-state needs (normalized cache, background refetch, offline queue, retries with backoff, invalidation graph). A server-state library (Phase 6 / 08) provides these; you can still use `useTransition` for UI integration, but `useActionState` alone does not replace a cache. **Another:** parallel independent uploads (the sequential queue would serialize them): use per-item state or a mutation library with concurrent mutations.

---

8. Best practices and anti-patterns

Best practices

1. **Return expected failures as state; throw only exceptional ones.** Validation errors → state; network catastrophes → boundary.
2. **Keep the real state authoritative.** The Action must update real state or revalidate before it ends; optimistic overlays are cosmetic.
3. **Make `submit` idempotent on the server** (idempotency keys) because Actions can be resubmitted (user double-click, retries, network replays).

4. **Provide `defaultValue` from returned state** to keep user input on failure.
5. **Use a shared `SubmitButton`** in the design system with `useFormStatus`.
6. **Prefer `use` for conditional context reads and promise props**, not as a general data-fetching strategy by itself; cache promises.
7. **Treat ref callbacks as lifecycle functions** and always use block bodies in TS.
8. **Centralize metadata** per route (framework conventions) and don't scatter conflicting `<title>`s.
9. **Preload critical resources** (LCP image, key fonts) via `preload`/framework features; avoid over-preloading.
10. **Guard `useEffectEvent`/`Activity` adoption** behind version checks (19.2+) and the `eslint-plugin-react-hooks` v6 rules.

Anti-patterns

Anti-pattern	Problem	Fix
Controlled inputs plus automatic form reset surprises	State and DOM diverge after reset	Choose: uncontrolled with `defaultValue` + action state, or controlled with explicit reset control (`form.reset()` or `requestFormReset`, which has no react.dev reference page)
Optimistic update without real state update	Flicker/rollback on success	Update state/revalidate within the Action
Throwing for validation	Error boundary replaces the form	Return errors as state
Giant `useActionState` reducers that fetch + mutate + derive	Hard to test	Keep the Action a thin orchestrator calling testable functions
Storing the promise in state created in render	Re-suspends	Create in loaders/server; cache by key
Treating `useFormStatus().data` as authoritative	It's the in-flight `FormData`, not saved data	Use returned state
Preloading everything	Competes with critical resources	Preload only LCP/critical path
Using `<Activity>` to cache heavy lists indefinitely	Memory growth	Limit hidden panes; unmount when stale

Justified exception: a controlled form with real-time validation may bypass automatic reset by managing state explicitly and not relying on uncontrolled semantics; use `useTransition` for pending and keep `onSubmit` with `preventDefault` if progressive enhancement isn't required.

---

9. Real-world application

Domain	Pattern	Constraints
E-commerce	Add-to-cart form Action with optimistic cart count; server recomputes totals	Inventory races; idempotency; price integrity; cart count reverts on failure with message
Dashboards	Inline-edit cells with Actions; optimistic cell value; undo	Concurrent edits (last-write-wins vs versioning); queueing semantics per cell

Domain	Pattern	Constraints
Finance	Money movement forms: **no optimistic UI** for irreversible operations; show pending and confirmed statuses	Idempotency keys; double-submit prevention; audit trail; strong server validation
Forms (long)	Multi-step wizard with Server Functions per step; `useActionState` for per-step errors	Persist partial progress; accessibility (focus first error, `aria-describedby`)
Auth	Login/logout Actions; redirect on success (framework helper)	Rate limiting; don't leak account existence; CSRF/origin checks
Realtime	Chat send with optimistic message; reconcile with socket echo	De-duplicate by client-generated id; ordering; failure state "retry"
Multi-team	Shared `SubmitButton`, `FormField`, error summary components using `useFormStatus`/Action state conventions	Documented contracts; testing helpers; version support (React 19 only)

10. Debugging and troubleshooting

Symptom A: "`isPending` never becomes true."

- **Investigation:** is the Action actually passed to React (form `action`/`startTransition`)? An `onSubmit` with `preventDefault` + plain `async` function isn't an Action. Is `useFormStatus` called inside the form?
- **Fix:** convert to `action={...}` or wrap in `startTransition`.

Symptom B: "Optimistic item flashes then disappears even though the request succeeded."

- **Root cause:** real state wasn't updated before the Action ended. **Fix:** update state/revalidate before returning; verify with logs in DevTools Components (real vs optimistic).

Symptom C: "Form fields clear after a validation error."

- **Root cause:** automatic reset after the Action. **Fix:** feed `defaultValue` from returned state (§5.3) or use controlled inputs.

Symptom D: "Second submit uses stale state / seems to ignore the first."

- **Investigation:** `useActionState` queue semantics; the second sees the first's result. If you expected independence, restructure.

Symptom E: "Warning: An optimistic state update occurred outside a transition or Action."

- **Fix:** call `addOptimistic` inside `startTransition`/form Action.

Symptom F: "`use` causes infinite suspense / repeated network calls."

- **Root cause:** unstable promise identity (created in render). **Fix:** cache/pass from parent/loader.

Symptom G: "TS error: Type 'HTMLElement' is not assignable to type 'void | (() => void | VoidOrUndefinedOnly)'." (ref callback)

- **Root cause:** arrow function with implicit return of the assignment. **Fix:** block body.

Symptom H: "`<title>` doesn't change / flickers between values."

- **Root cause:** multiple components render `<title>`, or a framework metadata API overrides it; check hoisting order. **Fix:** single source of truth per route.

Symptom I: "Stylesheet loads late, FOUC."

- **Investigation:** is `precedence` set on the `<link>`? Is it rendered in a Suspense boundary that already revealed? **Fix:** render stylesheets in the shell or the route component, set precedence; preload critical CSS.

---

11. Exercises

Level A — Knowledge

A1. Define "Action" in React 19 and list the three ways to start one. A2. Why can't `useFormStatus` report the status of a form rendered by its own component? A3. What happens to an optimistic overlay when an Action succeeds? When it throws? A4. List two restrictions on `use(promise)`. A5. What changed about ref callbacks' return value in React 19?

Level B — Code comprehension

B1. In §5.1, with a 300 ms action and a user clicking "Add" three times in 100 ms (button not disabled), what are the intermediate and final counts and the total time? B2. In §5.2, what does the user see if `addTodo` rejects after 1 s? B3. Explain why this ref callback is a bug under React 19 TypeScript types and what runtime behavior change it hints at:

```
<li ref={(el) => (nodes.current[i] = el)} />
```

Level C — Implementation

C1. Build a `LikeButton` with `useOptimistic` + `startTransition` that toggles instantly and reverts on failure with an error message. Include the real-state update. C2. Build a reusable `useFormAction` wrapper that combines `useActionState` and returns `{ action, state, isPending, errors }` for a Zod schema, with `defaultValue` preservation. C3. Implement a `PreloadLink` component that preloads a route's data/image on hover/focus using `preload` / `preconnect` and avoids duplicate preloads.

Level D — Debugging

D1. A form's submit button never shows a pending state though the action is async. Provide three possible causes and how you'd verify each. D2. After migrating to React 19, a gallery's refs object contains stale elements for removed items. Diagnose.

Level E — Architecture and engineering decisions

E1. Design the mutation architecture for a SaaS app: forms, inline edits, bulk operations, background jobs. Decide where Actions, TanStack Query mutations, and Server Functions are used and why. E2. A team wants to adopt `<Activity>` to preserve all routes' state. Evaluate risks and propose limits and metrics.

Level F — Expert challenge

F1. Specify a testing strategy for Action-based forms covering: pending UI, optimistic overlay and revert, queued submissions, error mapping, progressive enhancement (no-JS submission), and race conditions. Include what to test at unit, integration and e2e levels, and how to simulate slow servers deterministically.

---

12. Detailed answers and explanations

> Attempt the exercises first.

A1. An Action is a function (sync/async) run by React inside a transition so React can track pending state, error handling, optimistic updates, and form reset. Start with: `startTransition(fn)` / `useTransition`, `<form action={fn}>` / `formAction`, `useActionState` (returned `formAction` /dispatch).

A2. The hook reads the context of the nearest *parent* `<form>`; the component that renders the form is above it in the tree, not inside it. Use a child component or `useActionState`'s `isPending`.

A3. Success: overlay discarded when the Action settles; the UI shows real state (which should include the change). Throw: overlay discarded; the error goes to the boundary (or returned error state is shown); real state unchanged.

A4. Stable promise identity across renders (don't create in Client Component render); no try/catch to handle rejection (use error boundary or `.catch` upstream); call only during render (not in handlers/effects).

A5. A returned function from a ref callback is treated as a cleanup function. On unmount React calls the cleanup instead of `callback(null)`. Implicit arrow returns can accidentally return a non-function (TS error) or a function (unexpected cleanup).

B1. The queue runs sequentially: clicks at 0/50/100 ms; action 1 completes at 300 → count 1 (internally); action 2 starts with prev=1 → completes at 600 → count 2; action 3 completes at 900 → count 3. Final count 3 after ≈ 900 ms; `isPending` true throughout. Executed with three queued 30 ms actions: React rendered `count=0 pending=true` and then a single `count=3 pending=false`; the intermediate counts 1 and 2 were never committed (react.dev: multiple ongoing Actions are batched together), so the screen shows 0 → 3. The uncontrolled input resets once, after the last Action, not after each.

B2. Overlay shows the temp todo for ~1 s (dimmed), then the `catch` returns `{...prev, error}`; commit discards overlay; the todo vanishes; an alert shows "Could not add todo". The input has been reset (uncontrolled form) so the user's typed title is lost; improve by returning the attempted title in state and using it as `defaultValue`.

B3. The arrow returns the assignment's value (the element or `null`), which TS types reject because a ref callback may return only void or a cleanup function. Runtime change: under React 19, if the returned value were

a function it'd be treated as cleanup; also React won't call `ref(null)` when a cleanup is returned. Fix with a block body.

C1.

```
function LikeButton({ liked, count, toggle }: { liked: boolean; count: number; toggle: () =>
Promise<{ liked: boolean; count: number }> }) {
  const [real, setReal] = useState({ liked, count });
  const [optimistic, setOptimistic] = useOptimistic(real, (cur, _: void) => ({ liked: !cur.liked,
count: cur.count + (cur.liked ? -1 : 1) }));
  const [error, setError] = useState<string | null>(null);
  const [, startTransition] = useTransition();
  return (
    <>
      <button
        aria-pressed={optimistic.liked}
        onClick={() => startTransition(async () => {
          setError(null);
          setOptimistic();                              // overlay
          try { const next = await toggle(); startTransition(() => setReal(next)); }   // real
update (re-wrapped after await)
          catch { startTransition(() => setError("Could not update like")); }
        })}
      >
        {optimistic.liked ? "♥" : "♡"} {optimistic.count}
      </button>
      {error && <p role="alert">{error}</p>}
    </>
  );
}
```

Notes: after await, state updates are re-wrapped. Executed on 19.3.1: success showed `L 6` immediately and kept it; failure showed `L 6` during the Action and reverted to `U 5` with the error message. Rapid double-clicks queue overlays; for strict idempotency make `toggle` accept the desired value rather than "flip".

C2 (criteria). Wrapper takes `(schema, handler)`, returns an action `(prev, formData) => state`; validates via `schema.safeParse(Object.fromEntries(formData))`, returns `{ ok: false, values, errors }` or `{ ok: true, ... }`; exposes `getDefault(name)` from `state.values`; error mapping per field; tests for coercion and extra fields; accessibility attributes helper. Edge cases: multi-value fields (`getAll`), files, numeric coercion.

C3 (sketch). Use a module-level `Set<string>` for hints already requested (React dedupes anyway), call `preconnect(origin)` and `preload(url, {as})` in `onPointerEnter` / `onFocus` (event handlers, not render); if the route data is a fetch, call the router's prefetch. Cleanup is unnecessary for `preload`. Consider the cost on mobile data: only on explicit intent.

D1. (1) The handler isn't an Action (`onSubmit` + `preventDefault` + `await fetch` without transition) → no pending. Verify: does `<form action>` / `startTransition` appear? (2) `useFormStatus` is called in the form's own component. Verify by moving to a child. (3) The Action resolves immediately because the async work isn't awaited (missing `await` /returned promise). Verify with logs. (4) A returned `isPending` is read from a different `useActionState` instance.

D2. With cleanup-returning ref callbacks you must remove entries in the cleanup; if migrated code relied on `ref(null)` to delete entries, but the callback now returns a cleanup (or you return a value implicitly), React won't call with `null`; entries remain. Fix: delete in cleanup, or keep the `null` handling in the non-cleanup form but don't return a function.

E1 (criteria). Reasoned split: forms in RSC apps → Server Functions + `useActionState` (progressive enhancement, validation, revalidation); inline edits with frequent small mutations → TanStack Query mutations with optimistic updates (cache coherence) *or* Actions if framework revalidation is fast; bulk operations/background jobs → API that enqueues a job and returns an id; UI polls/subscribes; Actions only for starting. Cross-cutting: idempotency keys, error taxonomy, audit logging, optimistic only for reversible ops, testing. Evaluate on coherence and not mixing two caches for the same data.

E2 (criteria). Risks: memory (hidden trees), stale data in hidden panes (updates deprioritized), effects cleaned/re-run on reveal (analytics duplicates), accessibility (hidden content semantics), security (hidden UI of previous user after logout: must unmount on identity change), and version dependency (19.2+). Limits: whitelist routes with expensive-to-rebuild state; cap number of hidden activities; unmount on logout/tenant switch; metrics: memory, INP, reveal latency.

F1 (criteria). Unit: pure Action reducers (validation mapping, state transitions) without React. Integration (RTL): render form, deferred promise from a controllable mock, assert pending UI, optimistic overlay present, resolve/reject, assert revert/final; test queued submissions by calling submit twice; `act` + `await`; fake timers for delays. e2e (Playwright): real server with injected latency (route handlers delaying), no-JS test with `javaScriptEnabled: false` for progressive enhancement, race (submit twice, slow first). Determinism: controllable promises rather than timeouts; seed IDs; verify idempotency by replaying requests.

---

13. Senior engineer interview questions

Q1. "What is an Action and what problem does it solve?" *Strong answer:* A function run in a transition, giving React enough structure to manage pending, errors, optimistic state, and form reset uniformly. It replaces hand-rolled loading/error/rollback state machines and enables progressive enhancement when paired with Server Functions.

Q2. "How does `useOptimistic` revert, and why is that safer than manual rollback?" *Strong answer:* It's an overlay tied to the Action's lifetime; when the Action settles React renders the real state again, so success and failure both converge to the truth without manual undo code. The invariant is that real state must be updated by the Action.

Q3. "Why is `use` allowed in conditionals but hooks are not?" *Strong answer:* Hooks keep committed state in per-fiber slots matched by call order. `use(context)` is just a lookup, and `use(promise)` only tracks its thenables for the duration of a suspended render attempt (React's internal thenable state is not committed hook state), so call order across *committed* renders doesn't matter. It still must be called during render, and a conditional `use` that changes between attempts is something React 19.3 warns about in development.

Q4. "A form resets after every failed submit. How do you keep the user's input?" *Strong answer:* React resets uncontrolled forms after Actions by design. Return the submitted values in action state and bind them as `defaultValue`; or control inputs; or manage reset explicitly.

Q5. "How would you adopt React 19 refs and context changes in a library supporting 18 and 19?" *Strong answer:* Keep `forwardRef` for 18 or branch by version; for 19 accept `ref` as prop; ref callbacks must handle

both cleanup and `null` forms; use `Context.Provider` for 18 compatibility; test against both; communicate peer dependency ranges.

Q6. "When should you not use optimistic UI?" *Strong answer:* When actions are irreversible or have significant consequence (payments, deletions without undo, legal consents), when server validation commonly fails, or when showing a false state would mislead; show pending state instead.

---

14. Further learning resources

Resource	URL	Covers / why	Level	Section to study
React — `useActionState`	https://react.dev/reference/react/useActionState	Signature, queueing, permalink	Adv	Whole page, "Troubleshooting"
React — `useOptimistic`	https://react.dev/reference/react/useOptimistic	Overlay semantics, usage in Actions	Adv	Whole page
React — `useFormStatus`	https://react.dev/reference/react-dom/hooks/useFormStatus	Parent-form rule	Int	Caveats
React — `<form>`	https://react.dev/reference/react-dom/components/form	`action`, Server Functions, progressive enhancement, reset	Int–Adv	"Usage"
React — `use`	https://react.dev/reference/react/use	Promises, context, caveats	Adv	"Caveats", "Troubleshooting"
React — `useEffectEvent`	https://react.dev/reference/react/useEffectEvent	Effect events (stable since 19.2)	Adv	Whole page
React — `Activity`	https://react.dev/reference/react/Activity	Hidden-with-state (stable since 19.2)	Adv	Whole page
React — `preload` / `preinit`	https://react.dev/reference/react-dom/preload	Resource hint APIs	Int	Usage
React — Built-in `<link>` / `<title>` / `<meta>`	https://react.dev/reference/react-dom/components/link	Hoisting, `precedence`	Int	Special rendering behavior
React 19 upgrade guide	https://react.dev/blog/2024/04/25/react-19-upgrade-guide	Breaking changes (refs, TS, removed APIs)	Int	Whole page
React 19 announcement	https://react.dev/blog/2024/12/05/react-19	Feature overview	Int	Whole page

Links were fetched (HTTP 200) on 2026-10-22. Note that react.dev has no reference page for `requestFormReset` or `SuspenseList`.

15. Knowledge checkpoint

You should now understand

- Actions as transitions with tracked pending/error/reset; the queue semantics of `useActionState`.
- Optimistic UI as an auto-reverting overlay requiring real-state updates.
- `use` rules (stable promises, no try/catch, conditional OK) and the loop hazard.
- Ref-as-prop with cleanup, `<Context value>`, hoisted metadata, and resource hints.
- What `useEffectEvent` and `<Activity>` are for (both stable since 19.2).

You should be able to explain without notes

- The render sequence of an optimistic form submission, success and failure.
- Why `useFormStatus` must be in a child.
- Why validation errors should be returned, not thrown.

You should be able to implement

- Forms with `useActionState`, shared `SubmitButton`, optimistic list, promise-prop readers, resource preloading.

You should be able to debug

- Missing pending state, flicker on success, input loss, `use` loops, ref cleanup bugs.

Prerequisite for the next lesson (08 — React Compiler): many of the rules here (purity, no render side effects, stable identity) are exactly the rules the Compiler relies on to optimize your code automatically.